ARQUITECTURA DE REDES I / ARQUITECTURA DE REDES



GP24

Programación de sockets y SMTP

Esta práctica tiene un objetivo doble: por un lado, mejorar la comprensión del funcionamiento del protocolo de correo electrónico SMTP y, por otro, la programación de sockets en Python.

OBJETIVO

En esta práctica el alumno dispondrá de un ordenador con S.O. Linux y Python. El objetivo de la práctica es el envío de un correo electrónico ejecutando las órdenes del protocolo SMTP a través de un cliente TELNET y la programación de un cliente TCP en Python (en [1] puede encontrar un manual de referencia de programación en Python y en [2] una referencia a un manual de introducción a la programación usando Python).

No es necesario ser un experto en Python para hacer la práctica. Basta con saber programar en cualquier lenguaje y seguir las instrucciones de este documento. Para simplificar la tarea, se proporciona el esqueleto de un cliente TCP que establece una conexión contra un servidor. El estudiante sólo deberá completar la parte correspondiente al envío y recepción de mensajes.

Tal y como se ha visto en clase de Grupo Grande, el protocolo que se emplea para el envío de correos es SMTP (*Simple Mail Transfer Protocol*). Este protocolo de capa de aplicación trabaja sobre el protocolo de capa de transporte TCP. En esta práctica veremos cómo programar un cliente que establezca una conexión TCP con un servidor de correo sobre la que enviaremos los mensajes SMTP necesarios para enviar un correo. Se recomienda consultar la **sección 2.3.1** sobre correo electrónico del libro de texto. En concreto, en la página 99 podemos ver un ejemplo de una interacción SMTP.

El resto de este documento se estructura de la siguiente forma. En primer lugar, revisaremos el proceso de envío de un correo electrónico utilizando SMTP y un cliente TELNET. A continuación, se presentan los requisitos del cliente que queremos desarrollar. Finalmente, se proporciona y se comenta el esqueleto básico del programa a desarrollar para su finalización.

ENVÍO DE UN CORREO ELECTRÓNICO MEDIANTE UNA INTERACCIÓN SMTP USANDO "TELNET"

Como se ha visto en el Grupo Grande, podemos comunicarnos con un servidor de correo a través de la herramienta TELNET. El puerto que se utiliza por defecto para el servicio SMTP es el 25. En nuestro caso, trabajaremos contra un servidor de pruebas desplegado en un ordenador del laboratorio. Durante la sesión, el profesor facilitará la dirección IP y el puerto del servidor de correo electrónico.

Paso 1: Abrir un terminal y ejecutar la orden:

```
telnet IP_SERVIDOR PUERTO_SERVIDOR
```

Si se ha conectado correctamente aparecerá el siguiente texto:

```
220 NOMBRE_SERVIDOR Python SMTP proxy version 0.3
```

Paso 2: En la siguiente tabla se muestran una serie de órdenes del protocolo SMTP. Ejecute estas órdenes por orden, anotando las respuestas que recibe por parte del servidor y explicando su significado:

NOTA: En la secuencia 2, el correo a enviar debe realizarse desde la dirección puestoX@ar1.es, donde se debe cambiar la X por el número de puesto del alumno.

Secuencia	Petición	Respuesta	Comentario
1	`HELO NOMBRE_PUESTO`		
2	`MAIL FROM: puestoX@ar1.es`		
3	`RCPT TO: ar1@gpar1.es`		
4	`DATA`		
5	`Escribir un mensaje` `.`		
6	`QUIT`		

Paso 3: Avisar al profesor para confirmar que se ha recibido correctamente el correo electrónico en el servidor.

USANDO PYTHON PARA ENVIAR UN CORREO ELECTRÓNICO

El programa para desarrollar es un cliente TCP que se conecta al servidor SMTP utilizado en el apartado anterior. Si se realiza con éxito esa conexión TCP, el servidor nos devuelve un mensaje con el siguiente aspecto:

```
220 NOMBRE_SERVIDOR Python SMTP proxy version 0.3
```

En dicha línea, el código 220 indica el éxito de la conexión, siendo el resto de la respuesta un mensaje informativo acerca de ese código. El resto de respuestas SMTP sigue un formato similar: código 3 dígitos + frase para comprender qué significa ese código. En este apartado no se pide que se interprete dicho código, sino que se muestre por pantalla la respuesta del servidor tal cual la devuelve. Un cliente SMTP real interpreta ese código y toma una acción u otra en función del mismo.

Una vez tenemos establecida la conexión, el programa envía los mensajes descritos en la sección anterior (*HELO*, *MAIL FROM:, RCPT TO:, DATA*, *QUIT*) y espera la respuesta del servidor a cada uno de ellos mostrándola por pantalla. Los mensajes se introducen secuencialmente dentro del código fuente del programa. Para verificar que el cliente se ha ejecutado correctamente, deberemos comprobar que el correo electrónico se ha enviado correctamente (verificando los códigos de estado de las respuestas, por ejemplo) y que ha sido recibido en la dirección de correo del destinatario.

PUNTO DE PARTIDA

En el programa proporcionado `clienteSMTP.py` está definida la primera parte del programa (cómo crear el socket, cómo conectarse al servidor y cómo enviar la primera orden y recibir la primera respuesta). Sustituya

las **xxxxxxxx** por las funciones correspondientes y complete el código que falta para realizar el envío completo de un correo electrónico. En SERVER_IP y SERVER_PORT utilice los mismos valores utilizados en la conexión Telnet anterior.

```
#COMPLETAR LAS PETICIONES Y RESPUESTAS SMTP
# -*- coding: utf-8 -*-
from socket import *

#El servidor del correo al que nos vamos a conectar
SERVER_IP = 'XX.XX.XX.XX'   # Introducir la dirección del servidor que proporcione el profesor.
SERVER_PORT = XXXX          # Introducir el puerto del servidor que proporcione el profesor.

#El cuerpo del mensaje puede ser más largo, lo escribimos primero
#Si queremos añadir saltos de línea usamos \r\n
MENSAJE = 'Hola,\r\nEste es un correo de prueba\r\n'
#Para finalizar en envío de texto, se debe enviar "."
MENSAJE = MENSAJE + '\r\n.\r\n'

#Lista de tuplas (orden, código de respuesta) que se van a enviar
LISTA_ORDDENES = [('HELO PuestoXX\r\n', 250),
                  ('XXXXXXXXX', XXX),
                  ('XXXXXXXXX', XXX),
                  ('XXXXXXXXX', XXX),
                  (MENSAJE, 250),
                  ('XXXXXXXXX', XXX)]

#Establecemos conexión con el servidor
clientSocket = XXXXXX(AF_INET, SOCK_STREAM)
clientSocket.XXXXXX((SERVER_IP, SERVER_PORT))

#Recibimos respuesta
recv1 = clientSocket.XXXXXX(1024).decode('utf-8')
print(recv1)

#Por cada entrada de la lista, enviamos la orden, esperamos respuesta
#y procesamos la respuesta
for orden_actual in LISTA_ORDDENES:
    #PASO 1: enviar orden
    clientSocket.XXXXXX(orden_actual[0].encode('utf-8'))

    #PASO 2: recibimos respuesta y la imprimimos en pantalla
    recv1 = clientSocket.XXXXXX(1024).decode('utf-8')
    print(recv1)

    #PASO 3: procesamos la respuesta
    #comprobamos si el código de respuesta recibido se corresponde con el esperado
    if orden_actual[1] != int(recv1[:3]):
        #Si el código es diferente, se ha producido un error.
        #finalizamos bucle porque el proceso del envío del correo no se podrá completar
        print('Error: se ha recibido el código', recv1[:3], 'y se esperaba el', orden_actual[1])
        break
```

Notas importantes:

- **Recomendación: Para comprobar si el programa está funcionando correctamente puede reducir inicialmente el número de filas de la lista "LISTA_ORDENES" (no se enviará el correo electrónico pero permitirá comprobar si el resto de valores son correctos).**
- **Los caracteres "X" indican lugares en el código que deben ser completados, pero no se corresponden con un número concreto de caracteres.**
- **Los caracteres \r\n deben añadirse al finalizar una línea de texto que se quiera enviar (son equivalente a cuando se pulsa "enter" tras escribir la orden en el terminal.**
- Para indicar que un comando ha finalizado, debe incluir al final del mismo los caracteres CRLF, o, lo que es lo mismo \r\n.
- Para visualizar un mensaje por pantalla usamos el método: *print*.
- No olvide cerrar el socket al final del programa usando el método *close()*. `clientSocket.close()`.

EJECUTAR EL PROGRAMA

Una vez se haya terminado de completar el código, podemos probarlo. Python es un lenguaje interpretado así que no es necesario compilarlo. Se puede ejecutar de la siguiente forma. Abriendo un terminal y escribiendo lo siguiente:

```
python clienteSMTP.py
```

En condiciones normales, deberíamos obtener una salida similar a la siguiente:

```
250 NOMBRE_SERVIDOR

250 Ok

250 Ok

354 End data with <CR><LF>.<CR><LF>

250 Ok

221 Bye
```

REFERENCIAS

[1] Tutorial de Python. Disponible en: https://docs.python.org/es/3/tutorial/

[2] Documentación oficial del módulo socket: https://docs.python.org/3/library/socket.html